Trevon Woods
ITAI - 3377 - A.I. at the Edge
Sitaram Ayyagari
03/25/2026

Temperature Forecasting using Nixtla and Generative Models: A Comparative Study

Introduction

a. The main goal of this lab was to predict future temperature readings from IoT sensors by using one or many of Nixtla's timeseries forecasting techniques. The Nixtla library is known for its robust and user-friendly API. It enables the use of various forecasting models, including the advanced TimeGPT model used throughout the lab. The dataset that was used was sourced from kaggle. Using this sensor data I was able to achieve the objective of accurately forecasting temperature.

Data Preparation

Loading and Exploration

a. In preparation for training the model I needed to download the dataset from kaggle and load it into a pandas DataFrame. After creating my dataframe I did some initial exploration using `df.head()`, `df.info()`, and `df.describe()` to reveal:
   - **Columns:** `id, room_id/id, noted_date, temp, out/in`
   - **Data Types:**
     1. `temp` = Integer
     2. The rest of the columns were objects
   - **Missing Values:** There were no missing values in any of the columns which was confirmed by my using `df.isnull().sum()`. If there were any missing values They would have been filled in using the `.fillna(method='ffill', inplace=True)`
   - **Sample Distribution:** The `temp` column ranged from 21 to 51 degrees, with a mean of about 35 degrees.

Cleaning and Preprocessing

a. **Convert Datetime:** I began this step by first converting the `noted_date` column to a datetime object using the `pd.to_datetime()` with the format `'%d-%m-%Y %H:%M'`. This was needed in order for the model to interpret the date for forecasting.

b. **Indexing and Sorting:** In order to sort the data by the dates I had to set the `noted_date` column as the DataFrame index. This ensured that the data was in proper time series order.
c. **Column Dropping:** I dropped the `id` and `room_id/id` columns because they were irrelevant features.
d. **Resampling:** This was the last step of the cleaning steps. In order to handle any potential duplicate timestamps, the `temp` column was resampled to a 1-hour frequency by specifying `freq='H'.` This took the mean of the temperatures during each hour and filled any gaps introduced by resampling using forward-fill `(ffill)` followed by backward-fill `(bfill)` to make sure no missing data was uncaught.

Splitting and Scaling

a. **Data Splitting:** For the data split I chose to go with a 80/20 ratio for training and testing sets respectively. This resulted in a set of training entries with size `(2556, 1)` and a set of testing entries with a size of `(639, 1)`. These entries covered a period from `2018-07-28 07:00:00` to `2018-12-08 09:00:00`.
b. **MinMaxScaler:** I had to scale the temperature values between 0 and 1 because values in this range are more beneficial during training for many machine learning models. Also it prevents issues with differing scales. I instantiated the scaler and fit it on the training data only first then used it to transform both training and test sets.

Model Selection & Training

a. The model I decided to use for forecasting was Nixtla's TimeGPT model which is a new model in their library. It uses a form of AutoML but only for their GPT forecasting model with different parameters. It is a powerful and accessible solution for time series prediction. I used the following parameters when training the model:
    - **Horizon:** This is the number of days in the future that the model will predict. For this lab I set it to the length of the test set `(639 hours)`.
    - **Frequency:** This was inferred from the data, in the context of this lab the frequency was `'h'` for hourly.
    - **Confidence Intervals:** This was set to 80 and 90 and is used to define the prediction intervals (also known as confidence intervals) for probabilistic forecasting.
b. This model was trained initially without any additional features beyond the time index and the target variable itself, serving as a baseline.

Feature Engineering

a. To enhance the forecasting model's performance I engineered several time-based and lag features such as Hour of Day, Temperature Lag, Year, Month, and Day. The Hour of Day feature is extracted from the noted_date index, this feature captures daily cyclical patterns in temperature. The Temperature Lag is just the temperature reading from the previous hour. This provides the model with a form of attention that captures the dependency of the current temperature on its immediate past. This kind of understanding seems to be a strong predictor in time series forecasting. The Year, Month, and Day are simply there to capture longer-term seasonalities and trends.

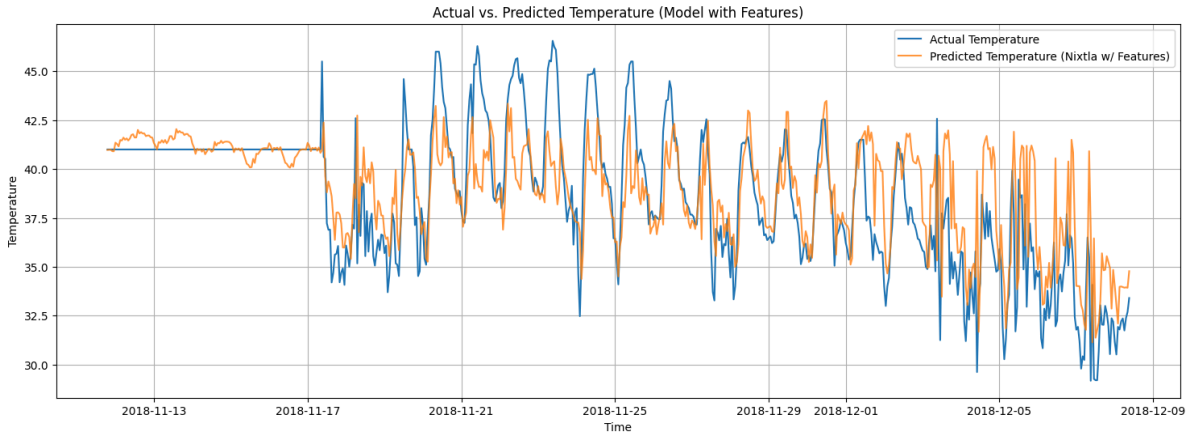
b. I chose these features because temperature tends to exhibit clear daily and seasonal patterns which makes its past values highly indicative of future values. There was one NaN value in the `temp_lag` feature which was dropped. I then re-split the dataset and re-scaled to finalize this new dataset. The training data (df) was prepared to include these features, forming `train_nixtla_input_fe`. Then I created another DataFrame X_df_nixtla to hold the features for both the training and prediction periods. This is done so the model has access to future feature values during forecasting.

Model Evaluation

Model with features

a. In this phase the TimeGPT model was trained with the engineered features. These were the evaluation metrics:
   - **Mean Absolute Error (MAE):** 1.7514
   - **Mean Squared Error (MSE):** 6.3368
   - **Root Mean Squared Error (RMSE):** 2.5173

b. These metrics depict that the model performs relatively well when predicting future temperatures. An MAE of 1.75 degrees tells me that, on average, the forecasted temperatures are within approximately 1.75 degrees of the actual temperature. This is a decent outcome given the typical temperature fluctuations. The RMSE penalizes larger errors more so 2.52 confirms that significant errors do not happen frequently.



c. The plot shows that the model's predictions closely match the actual temperature trend reasonably well even though there are some discrepancies. From my perspective this would more than likely be considered acceptable for most IoT temperature monitoring applications.

Rolling-Origin Cross-Validation

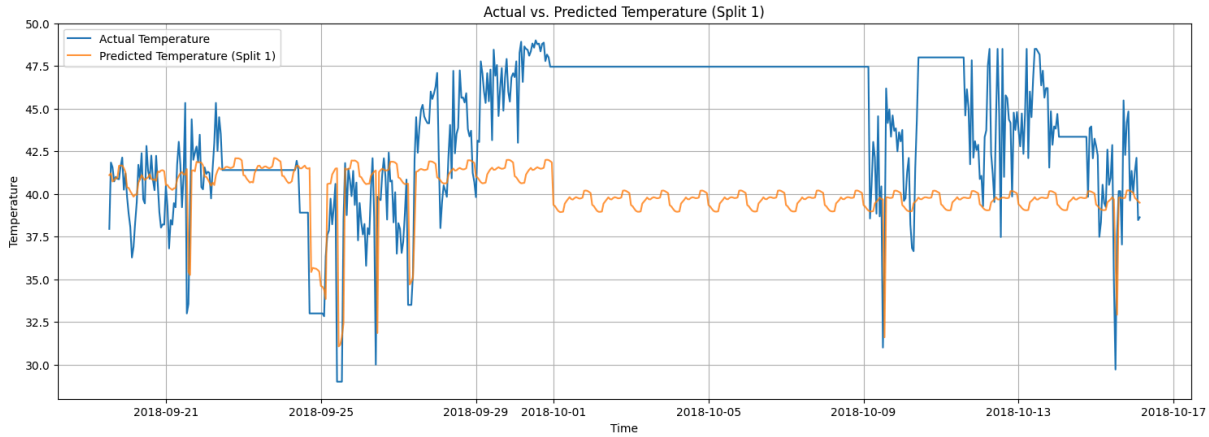
a. To further assess the robustness and generalization capability of the model, especially in the case of time series data, Rolling-Origin Cross-Validation (ROCV) was used. This is an evaluation strategy that simulates how a forecasting model would perform over time by repeatedly retraining and evaluating it on expanding windows of data. Here's how it's done:
    - **Split Data:** I declared `n_splits = 3` folds. I wanted to go higher but the model kept giving me errors saying that I had too few samples for a higher number of folds. In each fold:
        1. The training set expands, incorporating more historical data
        2. The test set remains a fixed size equal to the horizon of 639 hours which represents a future period to forecast.
    - Process:
        1. The model is trained on the first data split.
        2. Predictions are made for the subsequent fixed-size test period.
        3. Then the training data is rolled forward to include the previous test period, and the next test period is defined.
        4. Rinse and repeat steps 1-3 for all splits

Benefits of ROCV:

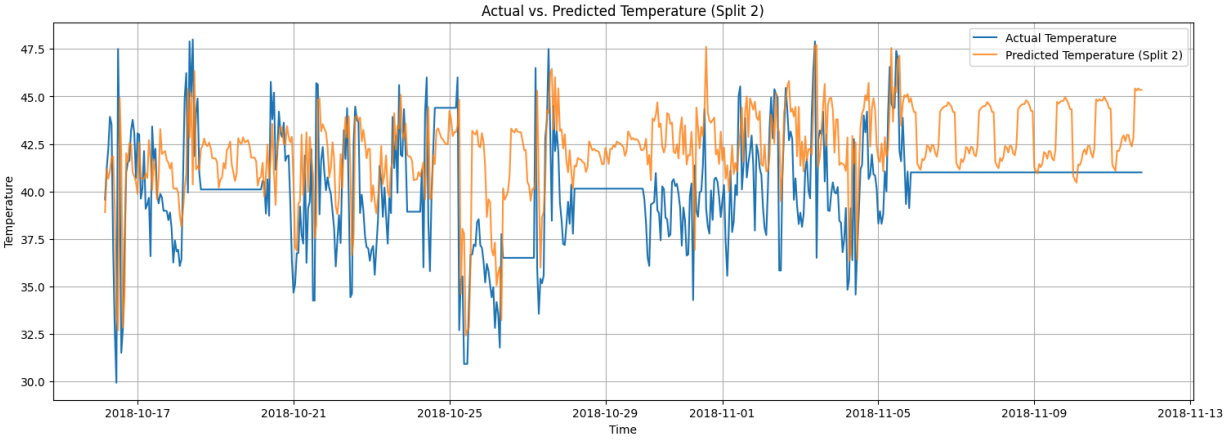
a. This strategy provides a more realistic estimate of model performance in a production environment where models are often retrained with new data. It also helps to analyse how stable the model's performance is across different time periods. This can reveal if it has an over sensitivity to specific patterns in a certain split.

These were the metrics for each split:

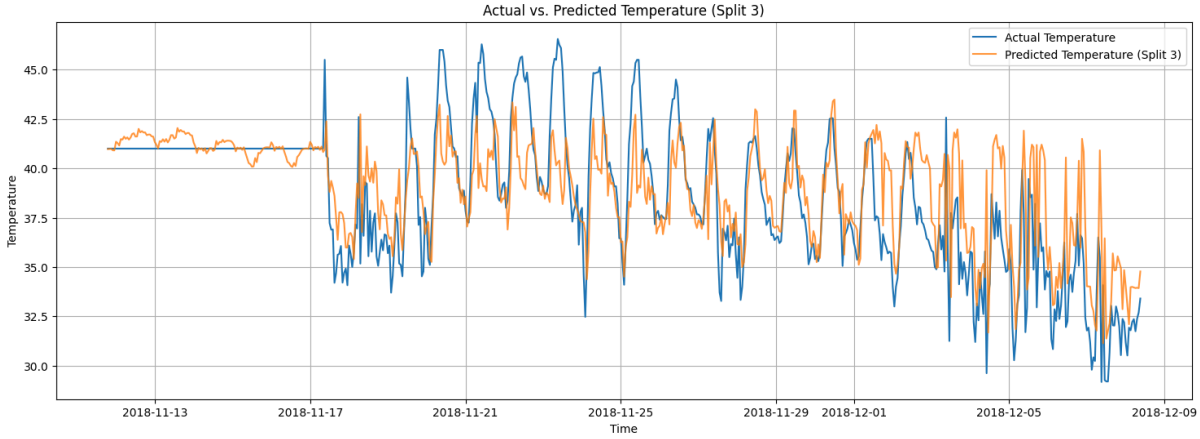
1. **Split 1:** MAE: 4.6655, MSE: 31.2378, RMSE: 5.5891



2. **Split 2:** MAE: 2.6491, MSE: 10.7914, RMSE: 3.2850



3. **Split 3:** MAE: 1.7514, MSE: 6.3368, RMSE: 2.5173



Generative Modeling

VAE Concept and Utility

a. A Variational Autoencoder (VAE) is a type of generative neural network capable of learning a compressed, latent representation of input data and then generating new, similar data from that latent space. Its utility in time series forecasting, particularly for data augmentation, stems from its ability to:
    - **Capture Data Distribution:** VAEs learn the underlying probability distribution of the input data, not just memorizing examples.
    - **Generate Novel Samples:** They can produce synthetic data samples that are similar to the training data but not exact copies, thus increasing the diversity of the training set.
    - **Anomaly Detection:** By quantifying the reconstruction error, VAEs can also be used to detect anomalous sequences.
b. In this project, the VAE was used to generate synthetic time series sequences of temperature, aiming to augment the original training data and potentially improve the forecasting model's performance, especially if the original dataset was small or had limited diversity.

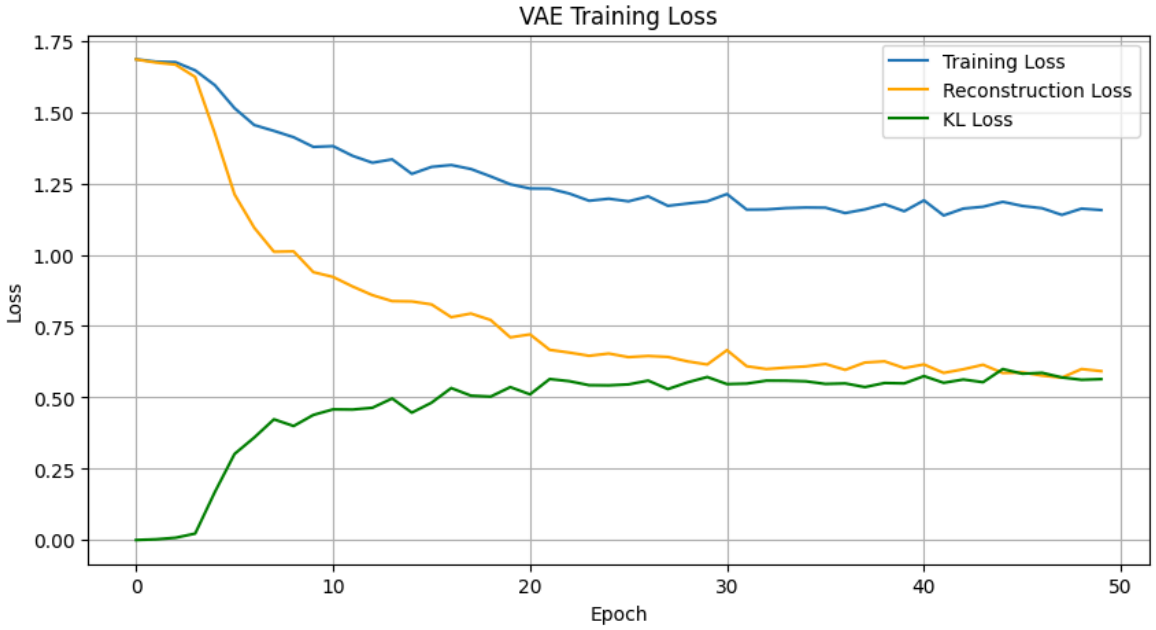
VAE Architecture and Training

a. **Encoder Network:**
    - **Input:** Sequences of scaled temperature data (e.g., sequence_length = 24 hourly data points).
    - **Layers:** Several LSTM layers (64, 32 units) followed by Dense layers. LSTMs are well-suited for capturing temporal dependencies in sequences.
    - **Output:** The encoder outputs two vectors: z_mean (mean) and z_log_var (log-variance), which define a Gaussian distribution in the latent space. A Lambda layer implements the reparameterization trick to sample a latent vector z from this distribution.
b. **Decoder Network:**
    - **Input:** A sampled latent vector z from the encoder's latent space.
    - **Layers:** A Dense layer, followed by a RepeatVector layer to expand the latent vector to the sequence_length, and then LSTM layers (32, 64 units) to reconstruct the sequence. A `TimeDistributed(Dense(1, activation='sigmoid'))` output layer is used to ensure the output matches the scaled input range (0-1).

Training Process

- The VAE was trained on sequences created from the scaled training temperature data. Each sequence had a length of 24 hours `(sequence_length = 24)`.
- **Epochs:** 50 epochs were used, with a `batch_size` of 128.
- **Loss Function:** The VAE minimizes a combined loss consisting of:

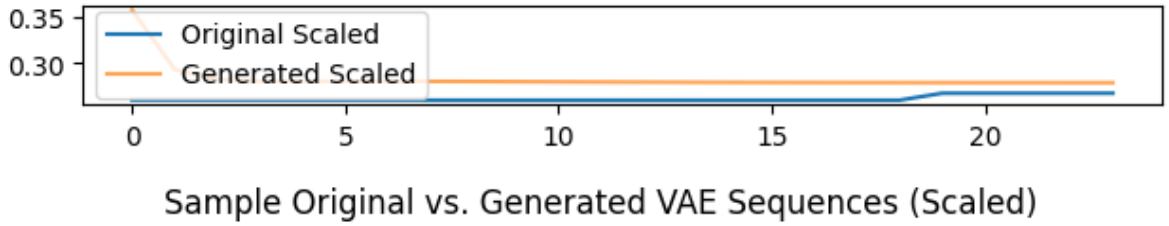
    - **Reconstruction Loss (MSE):** Measures how well the decoder reconstructs the input sequence from the latent representation.
    - **KL Divergence Loss:** Measures how much the learned latent distribution deviates from a standard normal distribution, encouraging a smooth and continuous latent space.
- **Optimizer:** Adam optimizer was used.
- The training history, including total loss, reconstruction loss, and KL loss, was plotted to monitor convergence and performance.



Synthetic Data Generation

- After training, the VAE's decoder was used to generate synthetic temperature sequences:
    1. **Sampling Latent Vectors:** Random points were sampled from a standard normal distribution in the latent_dim (8-dimensional) space.
    2. **Decoding:** These random latent vectors were fed into the trained decoder, which transformed them into new, synthetic scaled temperature sequences.
    3. **Flattening:** The generated sequences were flattened into a single, long 1D array of scaled temperature values.
- Approximately 60,768 synthetic data points were generated. These synthetic points were then concatenated with the original scaled training data, along with new timestamps, to create an augmented_nixtla_input DataFrame. The timestamps for the synthetic data were generated sequentially after the last original training data point, maintaining the hourly frequency.

Below is a sample plot comparing original and generated VAE sequences, illustrating how well the VAE learned to mimic the patterns of the real data:



Impact of Generative Models

To evaluate the impact of data augmentation, the Nixtla TimeGPT model was re-trained using the augmented_nixtla_input(original + synthetic data) as the training target. Crucially, in this experiment, the model was trained without explicit `X_df` features. The test set remained the same as in previous evaluations.

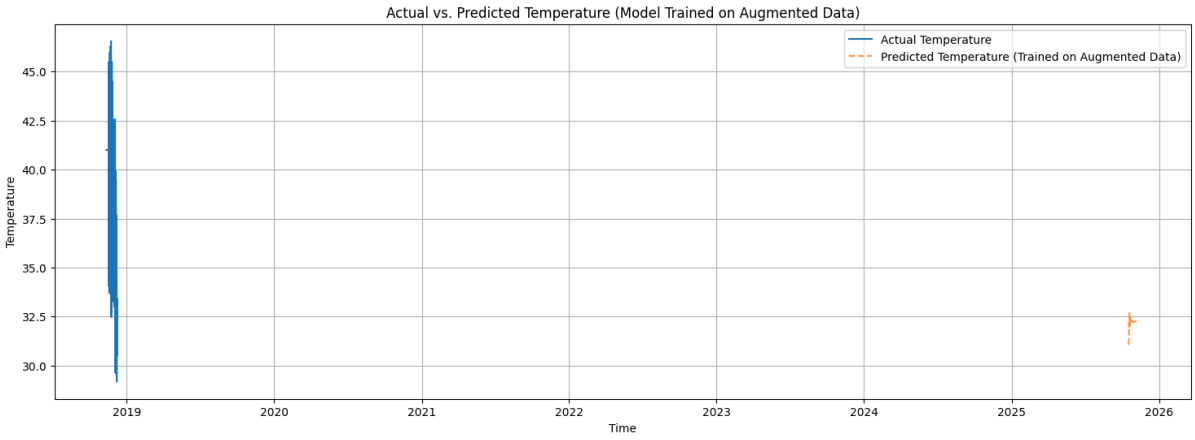
Metrics for Model Trained on Augmented Data

After training on the augmented data, the model's predictions were inverse-transformed to the original temperature scale, and metrics were calculated on the test set:

- **MAE (Augmented):** 6.4618
- **MSE (Augmented):** 53.0503
- **RMSE (Augmented):** 7.2836

Comparison with Feature-Based Model

Metric	Model with Features (No Augmentation)	Model Trained on Augmented Data	Change (Augmented - Features)
MAE	1.7514	6.4618	+4.7104
MSE	6.3368	53.0503	+46.7135
RMSE	2.5173	7.2836	+4.7663



Personal Reflection

Introduction

a. This Lab was a pretty challenging one but I got everything to work in the end. I've gotten the chance to use a really powerful model and learn some advanced strategies. I'll briefly touch on some of my challenges and key takeaways from completing this lab.

Challenges

b. My main challenge was with getting the dataset with engineered features to be accepted by TimeGPT. I had to really comb through the data and heavily debug my code in order to find the issue. Once I realised that it was a frequency problem I tried out all the different frequencies and settled on the hourly. Then I had an issue with the X_df features dataset because I was missing columns which initially went right over my head because I thought it was strictly to house the target data. Lastly, I had an issue with calculating KL Divergence for the VAE but that was quickly remedied after some research.

Key Take Aways

c. An interesting part of this assignment was the use of a VAE to generate synthetic temperature data to be used with the model. I have used VAE's for creating latent representations for image generation but never for samples of continuous values. This has broadened my viewpoint on just how powerful these models are. Another part of this lab that I will be looking into is how to get the VAE to generate better synthetic data. The augmentation significantly worsened the performance of the forecasting model. Both the MAE and RMSE increased substantially compared to the model without augmentation. I believe it was the fact that we did not include the engineered features into the synthetic dataset.

Conclusion

d. I learned a lot from this lab. I was able to explore temperature forecasting using a SOTA model created by Nixtla, TimeGPT. I was able to compare different approaches with and without engineered features. I was also able to view the impact of augmentation by generating synthetic temperature data using a Variational Autoencoder. Lastly, I was able to implement a strategy for training models on timeseries data called Rolling-Origin Cross-Validation. In the future I plan on trying to possibly add more features or do some hyperparameter tuning. I could even try using a different generative model like a Generative Adversarial Network (GAN) or a diffusion model.

References

- **Dataset Source:**
  - Kaggle: Temperature Readings IoT Devices
- **Python Libraries Used:**
  - pandas
  - numpy
  - scikit-learn
  - tensorflow
  - matplotlib
  - seaborn
  - kagglehub
  - nixtla
- **Nixtla Documentation:**
  - Nixtla Client Documentation
  - TimeGPT Documentation
- **TensorFlow / Keras Documentation:**
  - Keras Documentation